Fourth Implementation Iteration: One-Time Space Allocation

with Classroom, Laboratory, and MeetingRoom Resource Types

Software Design / Iteration Report

Name: Najib Attar
Date: March 2026

1. Iteration Goal

The goal of this fourth implementation iteration is to extend the prototype by adding a third concrete space resource type: `MeetingRoom`. The system already supports classrooms and laboratories through the general `Space` abstraction. In this iteration, the same request-processing mechanism is extended to operate on meeting rooms as well.

The request type and decision rules remain unchanged. A one-time request is evaluated using `AvailabilityRule` and `CapacityRule`, regardless of whether the requested resource is a classroom, laboratory, or meeting room.

2. Changes from Iteration 3

Compared with the third iteration, the following change was introduced:

- Added a new space type: `MeetingRoom`
- Updated the resource model so that the system now supports three concrete subclasses of `Space`:
  - `Classroom`
  - `Laboratory`
  - `MeetingRoom`

3. Scenario

A user submits a one-time request for a specific space resource, which may be a classroom, laboratory, or meeting room, for a given time slot and a specified number of participants.

The system checks whether the requested space is available during that time slot and whether the capacity of the space is sufficient for the participant count.

If both conditions are satisfied, an allocation is created. Otherwise, the request is rejected.

4. Modules and Their Tasks

A. Request Management Module

- Receive the one-time space request from the user
- Create and store `OneTimeRequest`
- Include request details:
  - requester
  - requested space
  - requested time slot
  - participant count
- Send the request to the Rule Evaluation Module

B. Resource Management Module

- Manage multiple space resource types
- Provide information for `Classroom`, `Laboratory`, and `MeetingRoom`
- Provide resource identity, name, type, and capacity

C. Rule Evaluation Module

- Evaluate the request using `AvailabilityRule`
- Evaluate the request using `CapacityRule`
- Check whether the requested space is already allocated during the requested time slot
- Check whether the space capacity is sufficient for the participant count
- Return the final decision: Approved or Rejected

D. Allocation Management Module

- Create `Allocation` if all rules pass
- Store the new allocation
- Update request status to Approved
- Reject the request if one or more rules fail
- Update request status to Rejected

5. Iteration Comparison

Iteration	Request Type	Resource Types	Rules Implemented
Iteration 1	OneTimeRequest	Classroom	AvailabilityRule
Iteration 2	OneTimeRequest	Classroom	AvailabilityRule + CapacityRule
Iteration 3	OneTimeRequest	Classroom + Laboratory	AvailabilityRule + CapacityRule
Iteration 4	OneTimeRequest	Classroom + Laboratory + MeetingRoom	AvailabilityRule + CapacityRule

6. Updated Class Diagram

Figure 1 shows the updated simplified class diagram for the fourth implementation iteration.

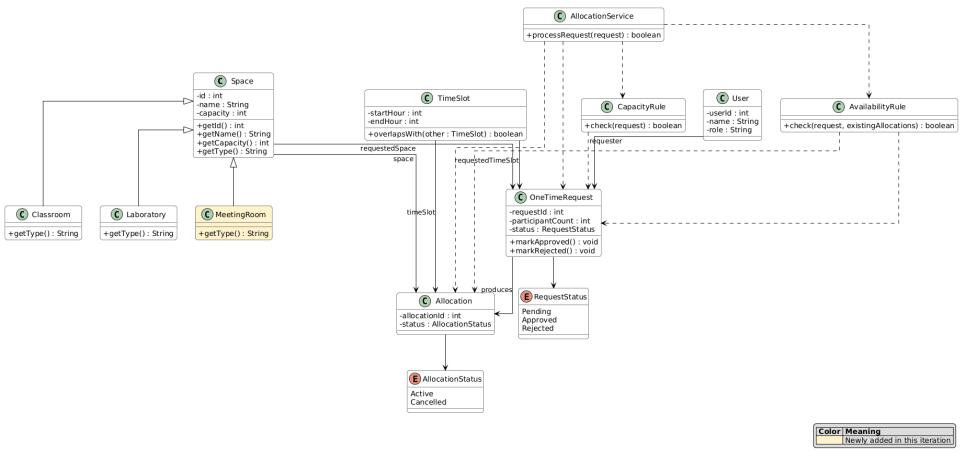

Figure 1: Updated Simplified Class Diagram

7. Updated Workflow Diagram

Figure 2 shows the workflow for the fourth iteration of the one-time space allocation process.

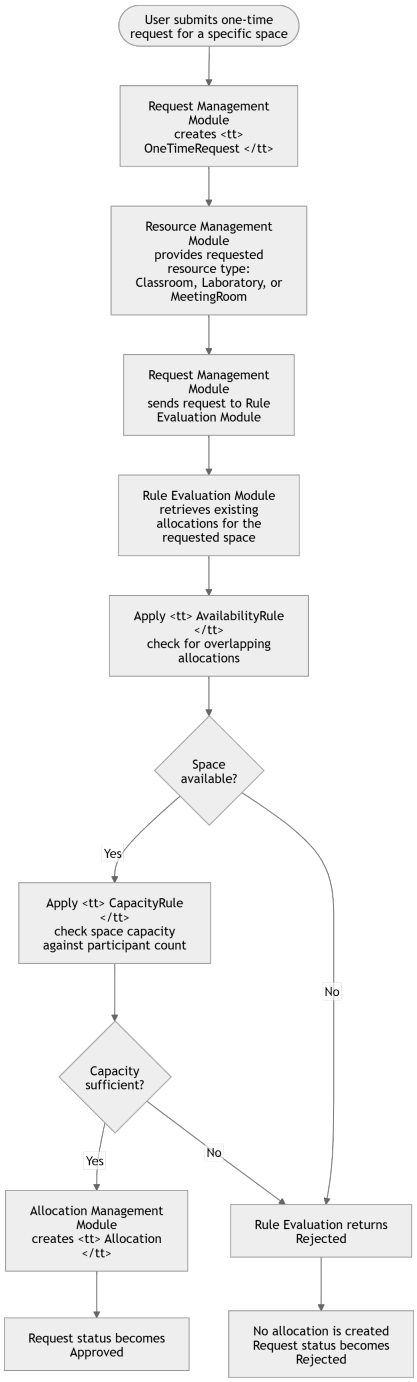

Figure 2: Updated Workflow Diagram

8. Prototype Implementation

To validate the design of this iteration, the C++ prototype was extended by adding a third concrete subclass of `Space`: `MeetingRoom`. Since the generalized resource model was already introduced in Iteration 3, the same request-processing and rule-evaluation logic could be reused without architectural changes.

The prototype was tested using scenarios involving classrooms, laboratories, and meeting rooms. These scenarios confirm that the system can process one-time requests consistently across all supported space types using the same allocation flow.

9. Summary

This fourth implementation iteration extends the prototype by adding support for a third space type while preserving the same request type and decision rules. The system now processes one-time requests for classrooms, laboratories, and meeting rooms.

This iteration demonstrates controlled iterative development by extending the generalized resource hierarchy without changing the overall allocation mechanism. Future iterations can continue this process by introducing additional rule types, request types, or more specialized resource constraints.